

Assignment 6 Part 2

Hey! You're still alive! Wow! I was thinking that the first part might have been too much. Anyway, there is more work for you to do now. Conceptually, all you have to do is implement two more public methods and add a little bit to a program I wrote called `SpeedTest.cpp`

Add to your header file and implement the following two methods:

- `void Remove(int data);`
- `~RedBlackTree();` // the destructor, maybe do this last

The remove algorithm is really complicated. I'm sorry in advance. You should write a lot of tests to make sure it's working right. Again, you can make design decisions and add private stuff to your red black tree as you see fit.

Resources:

Keep in mind that the visualizer, zyBook, and other resources might have slight differences in the remove algorithm. For example some compute the “in-order predecessor” instead of the “in-order successor” as is taught in the video lecture for the `remove()` operation. Your program should use the “in-order **successor**” algorithm.

It is required that you cite all substantial resources used via comments in your code.

Below are a couple resources you might find useful. Please cite these as well if you use them!

- <http://www.cs.toronto.edu/~wgeorge/csc265/2013/09/26/tutorial-3-red-black-tree-deletion.html>
- <https://www.usna.edu/Users/cs/crabbe/SI321/current/red-black/red-black.html>

Memory Management:

After `remove()` is done it would be wise to use it in the destructor. As you may have noticed I have written delete statements through-out the test cases as necessary. These are the tests of your destructor in conjunction with valgrind. You'll know you've done it all right when you can pass all of the tests and valgrind doesn't report any issues with anything. Your Red Black Tree should not leak any memory and there should not be any “invalid read” or “invalid write” warnings from valgrind. Consider my output (which includes even my secret grading tests!).

Sample Output:

```
user@machine$ make
g++ -Wall -g RedBlackTree.cpp RedBlackTreeTests.cpp -o rbt-tests
user@machine$ valgrind --leak-check=full ./rbt-tests
==74450== Memcheck, a memory error detector
==74450== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==74450== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==74450== Command: ./rbt-tests
==74450==
Testing Simple Constructor...
```

PASSED!

Testing Insert One Node...

PASSED!

Testing Insert Second Node...

PASSED!

Testing Insert Third Node...

PASSED!

Testing Insert Fourth Node...

PASSED!

Testing Insert Fifth Node...

PASSED!

Testing Insert Duplicates...

PASSED!

Testing Random Insert Stuff...

This test passes if it doesn't crash and valgrind reports no issues

PASSED!

Testing ToString Methods...

PASSED!

Testing Construction of Big Tree...

PASSED!

Testing Copy Constructor...

PASSED!

Testing Recursive FixUp...

PASSED!

Testing Remove Node With One Child...

PASSED!

Testing Remove Node With One Child...

PASSED!

Testing Remove Node With Two Children

PASSED!

Testing Remove (Double Black Condition)

PASSED!

Testing Remove Leaf...

PASSED!

Testing Remove Root As Only Node

PASSED!

Testing Contains...

PASSED!

```
Testing Get Minimum and Get Maximum...
PASSED!

Special Test One...
PASSED!

Special Test Two...
PASSED!

Special Test Three...
PASSED!

Special Test Four...
PASSED!

Special Test Five...
PASSED!

Special Test Six...
PASSED!

Different From Visualizer Tests...
PASSED!

ComprehensiveTest...
PASSED!

ALL TESTS PASSED!!
==74450==
==74450== HEAP SUMMARY:
==74450==    in use at exit: 0 bytes in 0 blocks
==74450==   total heap usage: 893 allocs, 893 frees, 131,523 bytes allocated
==74450==
==74450== All heap blocks were freed -- no leaks are possible
==74450==
==74450== For lists of detected and suppressed errors, rerun with: -s
==74450== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
user@machine$
```

Even as the professor for the course I'm proud of that! No memory issues whatsoever and it passes all the tests 100%. Also it's a functional red-black tree.

Now for the speed test program. A C++ red black tree is impressively fast, but maybe that's not apparent out of context. I wrote a simple program called `SpeedTest.cpp` that simulates a user ID allocation system. Imagine a large organization that has millions of members (users) that each need an ID (e.g., Twitter before it became $\mathbb{X}$). The program first creates a red black tree containing 3.7 million entries to simulate 3.7 million ID numbers.

On my machine this program runs in ~1 second. Wow, such fast. Your code, running on your machine, might be a little faster or slower. Anything less than 7 sec is acceptable.

Let's simulate the time it would take for users to leave or join. Suppose on any given day an average of 5 people leave the service (deleting 5 ID numbers), and 5 other people join the service (creating 5 ID numbers). Add code to my simulator to emulate this activity for one day.

- For people leaving, generate random ID numbers and if they're in the red black tree remove them until you've removed a total of 5.
- For people joining, do the same, but *add* them to the red black tree until you've added 5.
- Each time you do a `Remove()` or an `Insert()` measure the time the call takes on your system. You can use the same technique I used in the code given. You should print out 5 different measurements for `Remove()` and 5 different measurements for `Insert()`.

What kind of numbers do you see? About how fast is a `Remove()` from a red black tree? About how fast is an `Insert()` to a red black tree? Write down your answers to these questions as comments in your `Insert()` and `Remove()` methods in `RedBlackTree.cpp`

Now might be a good time to reflect. This is the end of the final assignment in CS3. What have you learned about CS since you started at F&M? You don't have to submit your answers to me. I just want you to think about it. If you don't think about it you can consider it a thought-crime.

Submission: Your program will be submitted using GitHub classroom.

- 1) Use the existing GitHub classroom repo you started in part 1 of this assignment.
- 2) Continue to edit and write code in the repo on your computer. Use `git add`, `git commit`, and `git push` to put that code onto github. I will have access to the repo. You can make as many commits as you like for this assignment with no penalty.
- 3) I will grade the final commit made before the due-date.
- 4) You don't have to make any submission on canvas.

Special Final Assignment Note: You cannot submit this assignment late, because it is due on the last day of class.